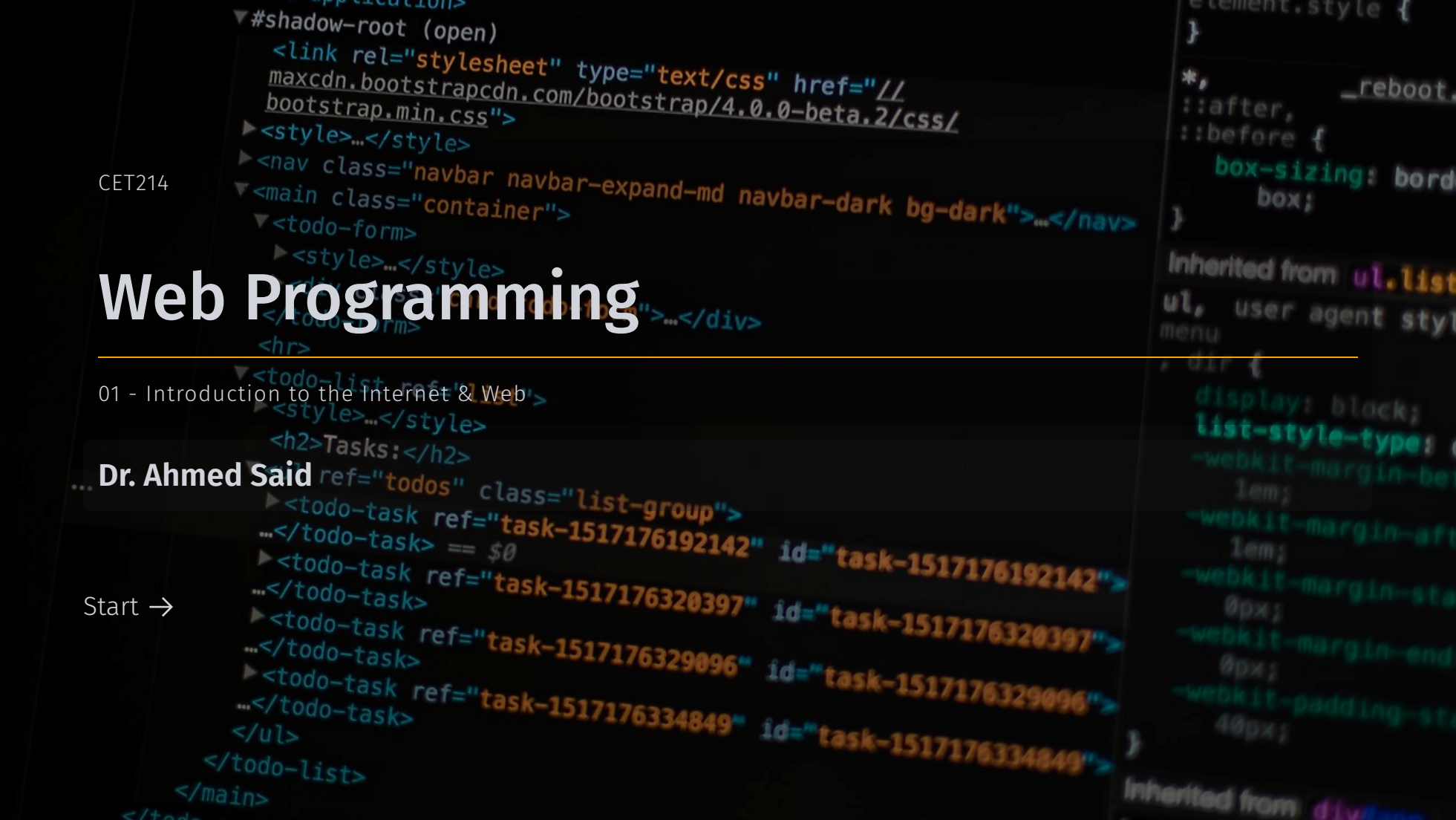
CET214

Web Programming

01 - Introduction to the Internet & Web

Dr. Ahmed Said

Start →



Learning Objectives

- Understand the architecture and history of the Internet
- Explain the client-server model and how web browsers work
- Describe the role of protocols (HTTP, TCP/IP, DNS)
- Read an HTTP request/response: method, status code, headers
- Trace what happens between typing a URL and seeing the page
- Identify key organizations that govern web standards
- Distinguish between the Internet and the World Wide Web
- Set up a basic web development workflow (editor, DevTools, Git)

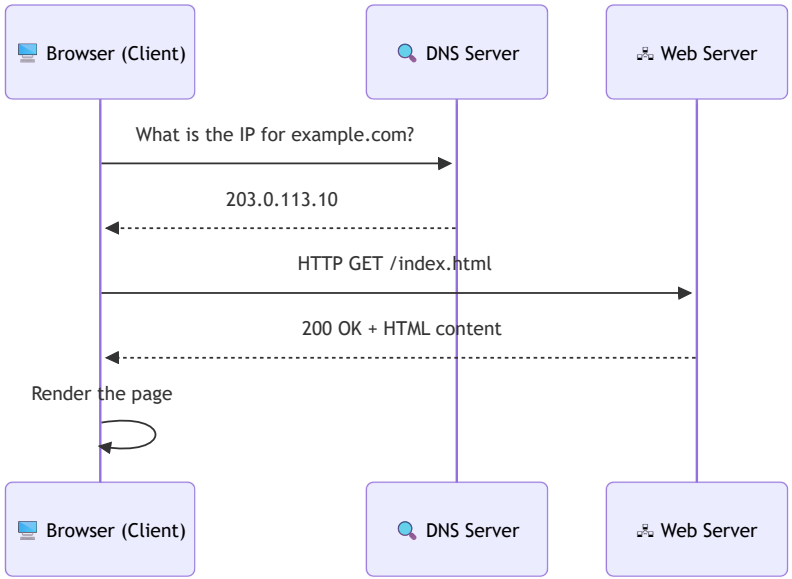
What is the Internet?

- A **network of networks**: tens of thousands of independently run networks (ISPs, universities, companies, governments) linking **billions of devices**
- Allows computers and devices to **communicate** using shared rules called **protocols**
- Every device is reached through an **IP address** (IPv4 `203.0.113.10` or IPv6 `2001:db8::1`)
- Built on layers of communication protocols:
  - IP → TCP/UDP → HTTP/SMTP/SSH/DNS...



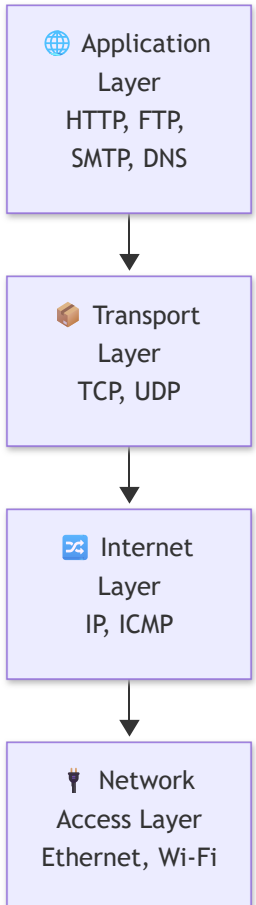
The Client-Server Model

Websites use a **client-server model** over the Internet.



- **Client**: The browser (Chrome, Firefox, Safari) that requests and displays web pages
- **Server**: A computer that stores, processes, and serves web pages
- **DNS**: Maps human-readable domain names to IP addresses

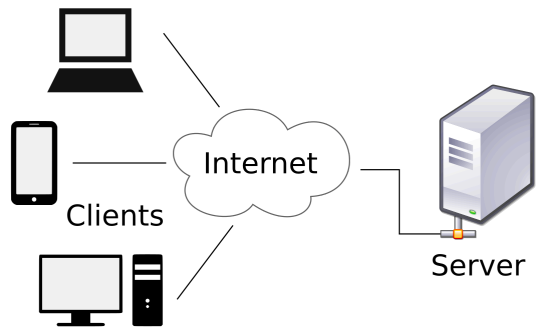
Layers of Protocol (TCP/IP Model)



- **Application Layer**: Protocols users interact with (HTTP for web, SMTP for email)
- **Transport Layer**: Reliable, ordered delivery (TCP) or lightweight, fast delivery (UDP; HTTP/3 runs over QUIC on UDP)
- **Internet Layer**: Handles routing and addressing (IP addresses like `192.168.0.1`)
- **Network Access Layer**: Physical transmission of data over cables, Wi-Fi, etc.

How the Web Works

- **IP addresses**: Numeric identifiers for every device — `192.168.0.1` is a private home-network address; public servers have public IPv4/IPv6 addresses
- **DNS (Domain Name System)**: Translates names like `google.com` to IP addresses `142.251.208.238`
- **URLs**: Say which resource to fetch — `https://example.com/about?lang=en`
- **Web servers**: Serve webpages to browsers over HTTP/HTTPS
- **Web browsers**: Request, receive, and display webpages



Anatomy of a URL

`https` `://` `example.com` `:8080` `/docs/intro.html` `?lang=en&page=2` `#setup`

SCHEME HOST PORT PATH QUERY FRAGMENT

Part	Answers	Notes
scheme	Which rules do we speak?	`https` , `http` , `mailto:` , `file:`
host	Which machine?	**DNS** turns this name into an IP address
port	Which door on that machine?	Usually omitted `80` for http, `443` for https
path	Which resource on the server?	Looks like folders; the server decides what it means
query	Any extra parameters?	`key=value` pairs joined by `&`
fragment	Which part of the page?	Scrolls to `id="setup"`

The one that surprises people

The **fragment** is never sent to the server - the browser keeps it and jumps to that element after the page loads. So `#setup` never appears in the server's logs.

Who "Runs" the Internet?

- **No single entity** controls the Internet
- Key organizations help maintain its infrastructure:
  - **ICANN** - coordinates domain names and IP address allocation (DNS root)
  - **IETF** - develops Internet protocol standards as RFCs (TCP/IP, HTTP, TLS)
  - **W3C** - develops web standards (CSS, accessibility/WCAG, many Web APIs)
  - **WHATWG** - maintains the **HTML Living Standard** and DOM (browser vendors)







Brief History of the Web

- **1969**: **ARPANET** the first packet-switched network (Internet ancestor)
- **1983**: **ARPANET** switches to **TCP/IP** - the birth of the modern Internet
- **1989**: Tim Berners-Lee proposes the World Wide Web at CERN
- **1991**: The first website (info.cern.ch) is opened to the public
- **1993**: **Mosaic** - the first popular graphical browser; CERN puts the Web in the public domain
- **1994**: W3C founded; Netscape Navigator released
- **1995**: Internet Explorer released; JavaScript created
- **2004**: **Web 2.0** era - Facebook, user-generated content
- **2004**: Firefox 1.0 released; AJAX popularized
- **2008**: Google Chrome released
- **2010s**: Mobile-first web, responsive design, SPAs; HTML5 (2014) → HTML Living Standard (2019)
- **2020s**: HTTP/3 (2022), AI-powered web, Web3 concepts, serverless architectures

Internet vs. World Wide Web

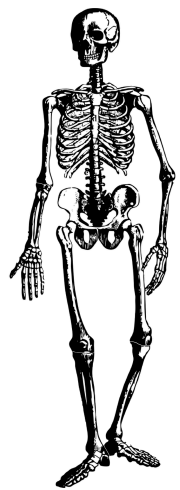
- The **Internet** is the global network infrastructure
- The **World Wide Web** is a service that runs on top of the Internet

Feature	Internet	World Wide Web
What	Network of networks	Collection of web pages
Protocol	TCP/IP	HTTP/HTTPS
Invented	1960s (ARPANET)	1989 (Tim Berners-Lee)
Includes	Email, FTP, VoIP, Web	HTML pages, CSS, JS
Access	Any network protocol	Web browsers

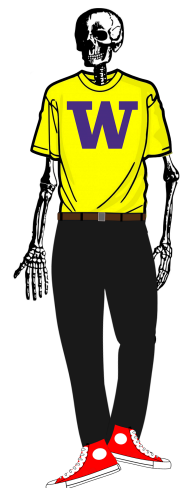
The Building Blocks of the Web



Content



Structure (HTML)

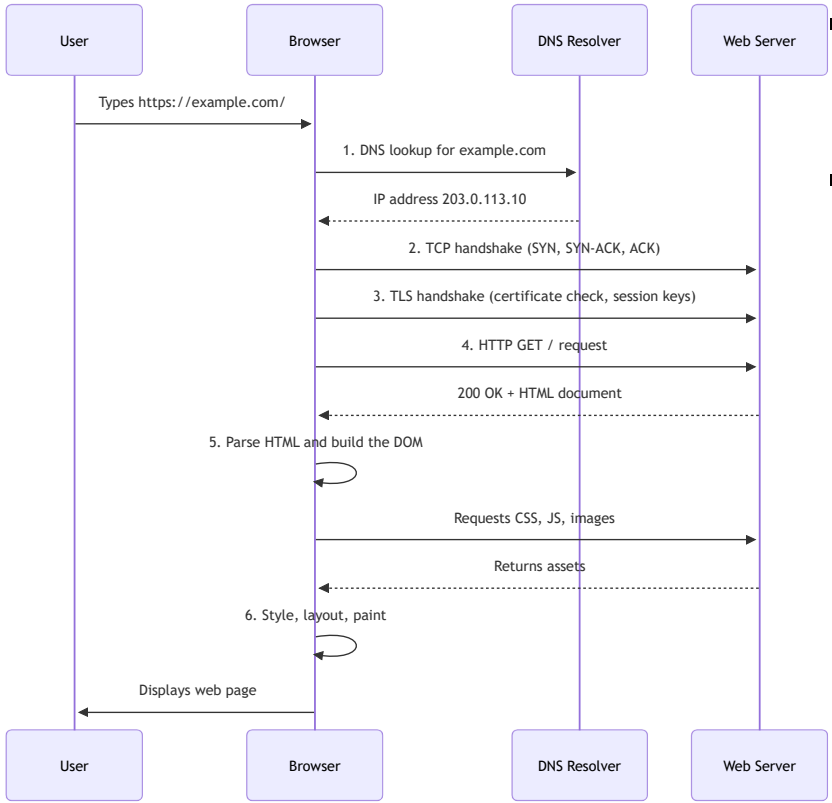


Style (CSS)



Behavior (JavaScript)

How a Web Page Loads: From URL to Pixels

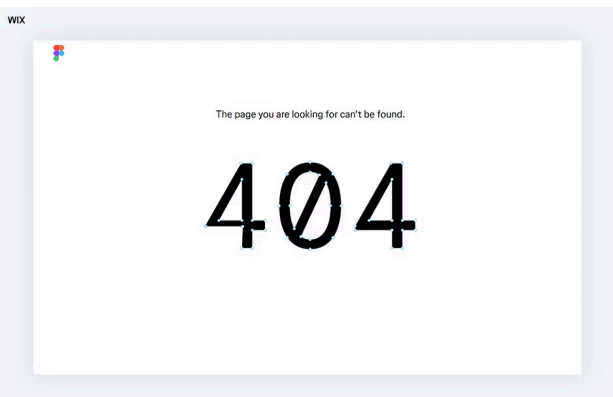


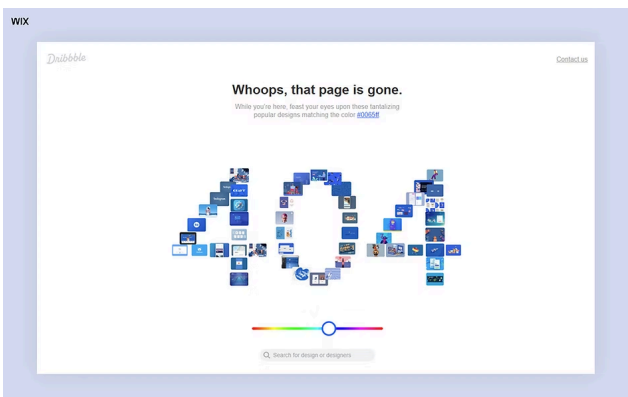
- **DNS** finds the server, **TCP** opens a reliable connection, **TLS** encrypts it, **HTTP** asks for the page, the **browser** renders it
- Browsers cache DNS answers and reuse connections, so later requests skip steps 1–3

⇄ Anatomy of an HTTP Request & Response

Status codes — the first digit tells you the story:

Class	Meaning	Examples
2xx	Success	`200 OK` , `201 Created` , `204 No Content`
3xx	Redirection	`301 Moved Permanently` , `304 Not Modified`
4xx	Client error	`400 Bad Request` , `403 Forbidden` , `404 Not Found`
5xx	Server error	`500 Internal Server Error` , `503 Service Unavailable`







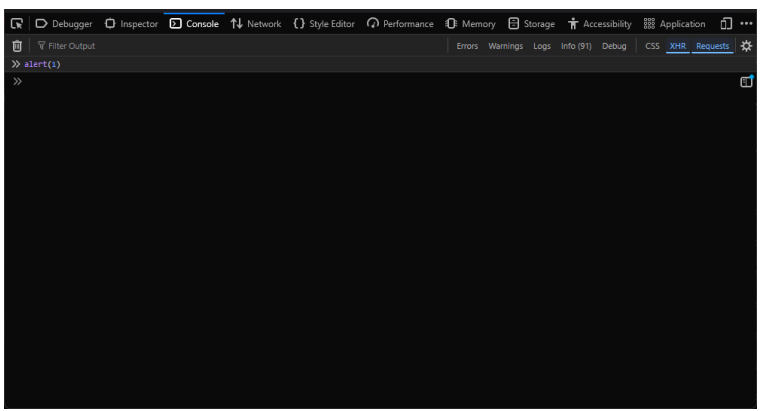
HTTP vs HTTPS

- **HTTP** (HyperText Transfer Protocol): The foundation of data communication on the web
  - Requests and responses are sent in **plain text**
  - Port **80** by default
- **HTTPS** (HTTP Secure): HTTP inside an encrypted **TLS** connection
  - **TLS** replaced the older, now-insecure **SSL** — people still say "SSL certificate"
  - Port **443** by default
  - Protects **confidentiality** (no eavesdropping), **integrity** (no tampering) and **authenticity** (you reach the real server)
  - Now the **standard** for all websites

> **Why HTTPS Matters**
>
> - Modern browsers mark HTTP sites as "Not Secure", and many features (geolocation, service workers, camera) only work over HTTPS.
> - HTTPS proves who you are talking to - not that the site is trustworthy.

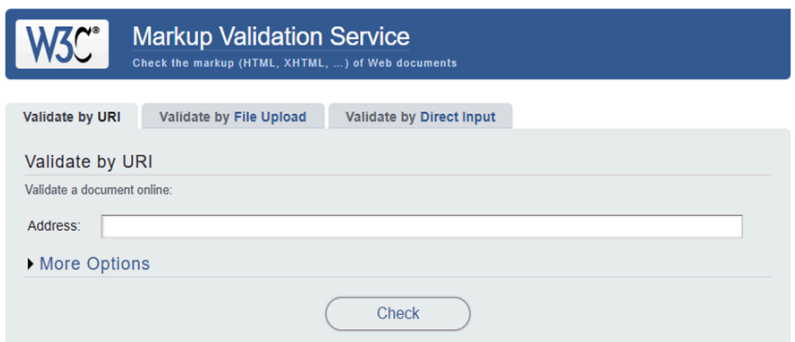
Developer Tools

- Every modern browser includes **Developer Tools** (DevTools)
- Open with `F12` or `Ctrl+Shift+I` (Windows/Linux) / `Cmd+Opt+I` (Mac) — or right-click → **Inspect**
- Key panels:
  - **Elements**: Inspect and edit HTML/CSS live
  - **Console**: Run JavaScript, view errors and logs
  - **Network**: See every HTTP request — method, status code, headers, timing
  - **Sources**: Debug JavaScript code
  - **Lighthouse**: Audit performance, accessibility and SEO



Your Development Setup

- **Editor**: **VS Code** — free, with Emmet built in (type `!` + `Tab` for an HTML skeleton)
- **Local server**: the **Live Server** or **Five Server** extension (auto-reloads the browser on save)
- **Browser**: Chrome, Firefox or Edge with **DevTools**
- **Validator**: validator.w3.org for HTML



Version control & publishing with Git and GitHub:

```
git clone https://github.com/<you>/web-programming.git
git add index.html
git commit -m "Add home page"
git push
```

> **Free Hosting**
>
> Enable **GitHub Pages** (repo **Settings** → **Pages** → **Deploy from branch** `main`) and your site is live at `https://<you>.github.io/<repo>/`.

Try It Yourself: Dissect a URL & a Request

1. Run the code, then change the URL and run it again. Which part is the **host**? the **path**? the **query**?

```
const url = new URL('https://example.com:8080/docs/intro.html?lang=en&page=2#setup');

console.log('protocol:', url.protocol);
console.log('host:', url.host);
console.log('path:', url.pathname);
console.log('query lang =', url.searchParams.get('lang'));
console.log('fragment:', url.hash);
```

2. Open any website, press `F12` → **Network**, reload. Click the first request and find its **method**, **status code**, and the `Content-Type` **response header**.

Evolution of the Web

Feature	Web 1.0 (1991–2004)	Web 2.0 (2004–present)	Web 3.0 (emerging)
Content	Static pages	User-generated content	Decentralized & semantic
Interaction	Read-only	Read & Write	Read, Write & Own
Technologies	HTML, CSS	AJAX, JS Frameworks, APIs	Blockchain, AI, dApps
Examples	Personal homepages, directories	Facebook, YouTube, Wikipedia	Ethereum, IPFS, NFTs
Data Control	Website owner	Platform companies	Users (via wallets & tokens)

What about Web 3.0?

"Web 3.0" is still evolving and debated — the term is used both for Tim Berners-Lee's **Semantic Web** and for **blockchain-based ownership** ("Web3"). You will research this further in the assignment!

Quick Quiz: How the Web Works

Q1. You open a link and get `404`. Whose "fault" is it: client or server? What about `503`?

✅ `404` is a **4xx client error** (the requested resource does not exist). `503` is a **5xx server error** (server overloaded or down).

Q2. Is email part of the World Wide Web?

✅ No — email (SMTP) is an **Internet** service. Webmail like Gmail is only a web interface to it.

Q3. Put in order: TLS handshake, render, DNS lookup, HTTP GET, TCP handshake.

✅ DNS lookup → TCP handshake → TLS handshake → HTTP GET → render.

Q4. Which organization maintains the HTML standard today?

✅ The **WHATWG** (HTML Living Standard), in agreement with the W3C.

Summary & Key Takeaways

- The Internet is a global network; the Web runs on top of it
- Web uses the **client-server model** with HTTP/HTTPS
- **TCP/IP** provides the layered communication framework
- **DNS** maps domain names to IP addresses
- Loading a page: **DNS → TCP → TLS → HTTP → render**
- HTTP messages have a **method**, **status code** (2xx/3xx/4xx/5xx) and **headers**
- **IETF** (protocols), **W3C** and **WHATWG** (web standards) and **ICANN** (names and numbers) keep the system working
- Websites are built with **HTML** (structure), **CSS** (style), and **JavaScript** (behavior)
- Your toolkit: **VS Code + Live Server**, **DevTools**, **Git/GitHub Pages**

Assignment

1. **Research**: Write a short summary (200-300 words) on the difference between the Internet and the World Wide Web
2. **Web 3.0**: Research Web 3.0 — what is it? How does it differ from Web 2.0? What technologies power it (blockchain, smart contracts, dApps)? Write a 200-word summary with your opinion on its future
3. **Explore**: Open DevTools (`F12`) → **Network** on any website and record 3 requests: method, status code, `Content-Type`
4. **Setup**: Create a GitHub account (if you don't have one), fork the course repository, and publish a one-line `index.html` with **GitHub Pages**

> `Course Repository`
>
> Fork and clone: **github.com/dr-ahmed-said/web-programming**

THANK YOU